Script de Présentation : Problème du Voyageur de Commerce (TSP)

Binôme Projet

Préparation

- Ouvrir **Pyzo** (ou VS Code).
- Ouvrir tous les fichiers dans des onglets.
- Avoir les images des résultats prêtes.

Phase 1 : Introduction & Contexte *0 :00 - 2 :00*

Contexte : Vous êtes debout, face au prof, pas de code à l'écran pour l'instant (ou juste le titre).

> *Vous :* "Bonjour. Nous allons vous présenter notre projet sur le **Problème du Voyageur de Commerce (TSP)**. Pour rappel, l'objectif est de trouver le cycle hamiltonien de coût minimal dans un graphe complet : visiter N villes et revenir au départ.
>
> Le défi majeur, c'est l'explosion combinatoire. C'est un problème **NP-complet**. Pour 5 villes, c'est trivial. Mais pour 15 ou 20 villes, le nombre de chemins possibles dépasse le milliard.
>
> Notre travail a donc consisté à implémenter et comparer deux familles de solutions :
>
> - Des **Heuristiques** (comme le glouton ou le 2-Opt) qui sacrifient l'exactitude pour la vitesse.
> - Une méthode **Exacte** (Branch & Bound) qui garantit la solution parfaite, mais au prix d'un temps de calcul exponentiel.
>
> "

Phase 2 : Architecture & Structures de Données *2 :00 - 5 :00*

Action

Ouvrez le dossier `structures/` et affichez `graphe_md.py`.

> *Vous :* "Avant même de parler d'algorithmes, parlons de la représentation des données, car c'est la clé de la performance. Nous avons fait un choix hybride.
>
> Premièrement, voici notre classe `GrapheMD`. C'est une **Matrice de Distances**. Pourquoi ? Parce que le TSP se joue sur un graphe complet. Les algorithmes comme le glouton ou le Branch & Bound passent leur temps à demander : "Quelle est la distance entre la ville A et la ville B ?". Avec une matrice `numpy`, cet accès se fait en temps constant $O(1)$, ce qui est critique pour ne pas ralentir les boucles imbriquées."

Action

Changez d'onglet pour `graphe_tl.py`.

Vous : "Par contre, nous avons aussi implémenté `GrapheTL`, qui est une **Liste d'Adjacence**. L'utilité ? Elle est spécifique à notre algorithme d'approximation (OptPrim). Cet algorithme travaille sur un **Arbre Couvrant Minimum (MST)**. Un arbre contient seulement $N - 1$ arêtes. Stocker un arbre creux dans une matrice $N \times N$ serait un gaspillage énorme de mémoire. La liste d'adjacence est ici beaucoup plus légère et permet un parcours en profondeur (DFS) linéaire."

Phase 3 : Les Algorithmes - Analyse du Code *5 :00 - 11 :00*

Vous : "Passons maintenant au cœur du moteur : les algorithmes."

1. Le Glouton : PPP (Algo_PPP)

Action

Ouvrez `algo_ppp.py`.

Vous : "Nous avons commencé par le plus intuitif : le **Plus Proche Voisin (PPP)**. C'est une heuristique constructive. On part d'une ville et on insère la ville non-visitée la plus proche. Regardez la boucle principale ici : on calcule le coût d'insertion à gauche ou à droite. Le problème : Il est "myope". Il fait des choix optimaux localement qui se révèlent désastreux globalement, créant des croisements d'arêtes."

2. La Star : OptPPP (2-Opt)

Action

Ouvrez `opt_ppp.py`. C'est le moment clé.

Vous : "C'est pour corriger cela que nous avons codé **OptPPP**, basé sur la recherche locale 2-Opt. Son principe est géométrique : un cycle optimal ne doit jamais se croiser. L'algorithme parcourt le cycle et détecte si deux arêtes (A, B) et (C, D) se croisent."

Action

Surlignez la ligne : `cycle[i+1:j+1] = reversed(cycle[i+1:j+1])`.

Vous : "Voici la ligne la plus importante du projet. Si on trouve un croisement avantageux à supprimer, on ne peut pas juste échanger deux indices. Il faut **inverser physiquement** toute la section du chemin entre les deux points d'échange. C'est ce que fait la fonction `reversed` ici. Cela "démêle" le nœud sans briser le cycle. Cette opération est répétée tant qu'il y a des améliorations possibles (`while amelioration`)."

3. L'Approximation : OptPrim

Action

Ouvrez `opt_prim.py`.

Vous : "Rapidement sur OptPrim. C'est une approche théorique garantie "2-approx". La stratégie est en 3 temps :

1. Calculer le MST avec Prim (nous utilisons un Tas binaire pour la rapidité).
2. Convertir le résultat en liste d'adjacence.
3. Faire un parcours DFS pour obtenir le cycle.

Bien que séduisant en théorie, nos tests montrent qu'il est souvent moins performant que le 2-Opt sur des points aléatoires."

4. L'Exactitude : HDS (Branch & Bound)

Action

Ouvrez `hds.py`. Restez calme, c'est le morceau complexe.

Vous : "Enfin, l'algorithme exact : HDS. Ici, on ne cherche pas "un" chemin, mais "le" meilleur. On explore l'arbre de toutes les permutations possibles. Mais pour ne pas explorer $N!$ solutions, on utilise le **Branch & Bound**."

Action

Montrez la fonction `calculer_borne_hds`.

Vous : "Tout repose sur cette fonction de **Borne Inférieure**. Pour chaque nœud partiel, on estime le coût minimal qu'on pourrait atteindre. On utilise la "Demi-somme des 2 arêtes les moins chères". Si cette estimation dépasse déjà le meilleur coût complet qu'on a trouvé ailleurs..."

Action

Montrez le code : `if noeud.bound >= cout_minimal: continue`.

Vous : "... alors on coupe la branche immédiatement (**Pruning**). C'est cet élagage qui nous permet de résoudre des instances jusqu'à 15 villes en un temps raisonnable, là où un Brute Force planterait à 12."

Phase 4 : Résultats et Analyse Critique *11 :00 - 14 :00*

Action

Affichez vos images de graphiques (stats et courbes).

Vous : "Nous avons validé nos algos sur 100 essais aléatoires. Voici les conclusions :

Sur la qualité (Graphique de gauche) :

- HDS est la référence (erreur 0%).
- OptPPP est bluffant : Il arrive en moyenne à moins de 1% de l'optimal. Il corrige presque toutes les erreurs du PPP.
- OptPrim est en retrait (+17% d'erreur), confirmant que l'approche MST est moins adaptée aux graphes euclidiens aléatoires.

Sur le Temps (Graphique de droite - Échelle Log) : C'est ici que se joue la limite du HDS. Regardez la courbe orange : c'est une droite sur échelle log, donc une exponentielle. Pour $N = 10$, il met 60ms. Pour $N = 15$, quelques secondes. Pour $N = 20$, il ne finirait pas avant des heures. À l'inverse, OptPPP (en bleu) reste instantané quelle que soit la taille N."

Phase 5 : Conclusion et Démo Live *14 :00 - 15 :00*

Action

Lancez votre terminal : `python main.py`.

Vous : "Pour conclure, ce projet nous a montré qu'il n'y a pas d'algorithme parfait, mais des compromis. Si je dois livrer 15 colis, j'utilise HDS pour économiser le moindre

kilomètre. Si je suis Amazon avec 1000 colis, j'utilise OptPPP pour avoir une réponse immédiate et quasi-parfaite."

Action

Lancez une démo rapide sur $N = 15$ villes.

Vous : "Je lance une démo sur 15 villes... Vous voyez, OptPPP a fini instantanément. HDS réfléchit... il élague l'arbre... et voilà, il trouve la solution optimale, souvent identique ou très proche de celle d'OptPPP.

Merci de votre écoute, nous sommes prêts pour vos questions."